Supplementary Data

—

GABAC: an arithmetic coding solution for genomic data

Jan Voges[1,*,†], Tom Paridaens[2,*,†], Fabian Müntefering[1], Liudmila S. Mainzer[4,5], Brian Bliss[4], Mingyu Yang[3], Idoia Ochoa[3], Jan Fostier[2], Jörn Ostermann[1], and Mikel Hernaez[5,*]

[1]Institut für Informationsverarbeitung (TNT), Leibniz University Hannover, 30167 Hannover, Germany
[2]IDLab, Ghent University - imec, 9050 Ghent, Belgium
[3]Electrical and Computer Engineering, University of Illinois at Urbana-Champaign, Urbana, IL 61801, USA
[4]National Center for Supercomputing Applications, University of Illinois at Urbana-Champaign, Urbana, IL 61801, USA
[5]Carl R. Woese Institute for Genomic Biology, University of Illinois at Urbana-Champaign, Urbana, IL 61801, USA
[†]Joint first authors.
[*]To whom correspondence should be addressed.

Supplementary Note 1: Data

To evaluate the performance of GABAC we generated a set of what we call *descriptor stream files*:

- In a first step (see subsection A) we selected initial data from the MPEG-G Genomic Information Database (https://mpeg-g.org).
- If necessary we subsequently aligned the data following the GATK Best Practices (https://software.broadinstitute.org/gatk/best-practices/workflow?id=11165) and extracted chromosome 11 from the data (see subsection B).
- In a final step we used our modified versions of the genomic codecs CRAM and DeeZ to process the data. These modified tools output their internal data representations, such as mapping positions and read pairing information, to separate descriptor stream files (see subsection C).

The final simulation data is summarized in subsection D.
The scripts used to perform the operations described in the following text can be found in the GABAC repository (https://github.com/mitogen/gabac).

A. Data Origins. For our simulations we selected files pertaining to the items 01 and 02 from the MPEG-G Genomic Information Database. Table 1 shows the details of the selected files.

Table 1. Files selected from the MPEG-G Genomic Information Database.

ID	File Name(s)	Experiment Type	Species	Sequencing Technology
01	ERR174310_{1|2}.fastq ... ERR174314_{1|2}.fastq	WGS	*H. sapiens*	Illumina HiSeq 2000
	Origin: http://www.ebi.ac.uk/ena/data/view/ERP001775			
02	NA12878_S1.sam	WGS	*H. sapiens*	Illumina HiSeq 2000
	Origin: http://www.ebi.ac.uk/ena/data/view/ERP001960			

From item 01, we decided to use the file pairs ERR174310 through ERR174314, as shown in Table 1. We concatenated the files with the following commands:

```
$ cat ERR174310_1.fastq ERR174311_1.fastq \
      ERR174312_1.fastq ERR174313_1.fastq \
      ERR174314_1.fastq >ERR17431_1.fastq
$ cat ERR174310_2.fastq ERR174311_2.fastq \
      ERR174312_2.fastq ERR174313_2.fastq \
      ERR17431_2.fastq >ERR17431_2.fastq
```

B. Alignment. We aligned the FASTQ files pertaining to item 01 before further processing. We conducted this procedure as recommended in the GATK Best Practices.
The following tool versions were used for the alignment process:

- BWA 0.7.13 (1)
- GATK 4.0.8.1 (2)
- Picard 2.18.14 (https://broadinstitute.github.io/picard/)
- Samtools 1.3 (built with HTSlib 1.3) (3)

As reference we used the file `human_g1k_v37.fasta` from the GATK resource bundle version 2.8 (ftp://ftp.broadinstitute.org/bundle/b37).
The file `NA12878_S1.sam` from item 02 is already aligned to another reference (Genome Reference Consortium human build 37 (GRCh37), also known as hg19).
Finally, we extracted chromosome 11 from both resulting SAM/BAM files.

C. Generation of descriptor stream files with CRAM and DeeZ. For the generation of the descriptor stream files we used our modified versions of CRAM and DeeZ to process the data. These modified tools output their internal data representations, such as mapping positions and read pairing information, to separate descriptor stream files.

C.1. CRAM. Here we use our modified version of HTSlib (https://github.com/samtools/htslib) that, instead of producing a single CRAM (4, 5) file during the SAM/BAM-to-CRAM conversion, writes each occurring type of internal stream (such as mapping positions and read pairing information) uncompressed to a separate file. This way, we are able to employ GABAC in addition to the built-in entropy coding mechanisms from the CRAM implementation in HTSlib.
For our simulations we used Samtools 1.9 (https://github.com/samtools/samtools). We linked Samtools to our modified HTSlib which is available at https://github.com/voges/htslib/releases/tag/1.9-rawstreams. Our modified HTSlib is based on HTSlib 1.9.
We generated the CRAM streams with the following command:

```
$ samtools view -C alignments.sam -T reference.fasta -o alignments.cram
```

Table 2 summarizes the semantics of the different CRAM descriptor streams. For more detailed information on the CRAM descriptor streams we refer the reader to the CRAM specification (https://samtools.github.io/hts-specs/CRAMv3.pdf).

Table 2. CRAM descriptor stream semantics.

Stream	Semantics
BF	Bit flags
AP	In-seq positions
FP	In-read positions
RL	Read lengths
DL	Deletion lengths
NF	Number of records to next fragment
BA	Bases
QS	Quality scores
BB	Stretches of bases
QQ	Stretches of quality scores
FC	Read features codes
FN	Number of read features
BS	Base substitution codes
IN	Inserted bases
RG	Read groups

Stream	Semantics.
MQ	Mapping qualities
TL	Tag IDs list
RN	Read names
NS	Next fragment reference sequence ID
NP	Next mate alignment start
TS	Template size
MF	Next mate bit flags
CF	Compression bit flags
TM	Test mark (for debugging)
RI	Reference id from BAM header
RS	Reference skip length (N read feature)
PD	Number of padded bases
HC	Number of hard clipped bases
SC	Soft clipped bases

C.2. DeeZ. Here we use our modified version of DeeZ (6) that writes each occurring type of internal stream (such as mapping positions and read pairing information) uncompressed to a separate file. This allows us (like for CRAM above) to use GABAC instead of the internal entropy codecs of DeeZ. Our modified version of DeeZ is based on DeeZ 1.1. It is available at https://github.com/voges/deez/releases/tag/v1.1-rawstreams.
We generated the DeeZ streams with the following command:

```
$ deez alignments.sam -r reference.fasta -o alignments.deez
```

Table 3 summarizes the semantics of the different DeeZ descriptor streams. We deducted this information from the DeeZ source code.

D. Final Simulation Data Set. The set of descriptor stream files generated by CRAM and DeeZ contained some empty descriptor stream files which we removed before further processing. Moreover, for the CRAM descriptor stream files, we omitted all descriptor streams not listed in the official CRAM specification, i.e., files having the following extensions: `.cram_raw`, `.DS_Cram_Info`, `.DS_<3-lettercode>`, `.DS_<number>`. In summary this process yielded:

- 28 CRAM descriptor stream files pertaining to item 01, chromosome 11;
- 37 CRAM descriptor stream files pertaining to item 02, chromosome 11;
- 39 DeeZ descriptor stream files pertaining to item 01, chromosome 11;
- 52 DeeZ descriptor stream files pertaining to item 02, chromosome 11.

From these $28 + 37 = 65$ CRAM descriptor stream files and $39 + 52 = 91$ DeeZ descriptor stream files we omitted 7 files and 20 files, respectively, because they were too small to provide reasonable performance numbers. Finally, this yielded 58 CRAM

Table 3. DeeZ descriptor stream semantics.

Stream	Semantics
EditACGT_IS_NVec	Inserted N-nucleotide flags
EditACGT_IS_SeqVec	Inserted nucleotides
EditACGT_NVec	Mismatched N-nucleotide flags
EditACGT_SeqVec	Mismatched nucleotides
EditACGT_U_NVec	Unmapped N-nucleotide flags
EditACGT_U_SeqVec	Unmapped nucleotides
EditHSlen	Size of hard-/softclips
EditLen	Size of operations excluding H/S/X
EditLocation	Delta encoded location of edit < 255
EditOpcodes	CIGAR opcode sequence
EditOPLen	Opcode length per read
EditSeqend	Length of sequence
EditSeqpos	Position of CIGAR in sequence
EditStitch	Delta encoded location of edit >= 255
EditXlen	Size of X opcodes
MappingFlag	Mapping flags
MappingQual	Mapping qualities
PairedChr	Chromosome IDs
PairedDiff	How many reads to look back for paired read
PairedTlen	TLEN size
PairedTLenBit	If the read is matched with a pair
Quality	Quality scores
ReadNameContent	Read name token data
ReadNameIndex	List of tokens
ReadNamePair	Flag if read name is paired
SequenceFixes	Delta encoded location of patched nucleotide < 255
SequenceFixesSt	Delta encoded location of patched nucleotide >= 255
SequenceReplace	Patched nucleotides
Tag[Name]	Data for optional tag [Name]
TagBuffer	Basic tag data
TagLib	Tag names / indices

descriptor stream files and 71 DeeZ descriptor stream files. To reduce the total test set footprint, all remaining descriptor stream files were limited to 200 MiB. As a result, 18 files were capped at 200 MiB. This approach allows for the analysis of a more extensive test set, while preserving a reliable representation of the coding performance for each of the compared tools. The total size of the resulting test set, consisting of 129 files, is approximately 8.9 GiB.
The final simulation data set is available at https://www.tnt.uni-hannover.de/staff/voges/data/gabac/.

Supplementary Note 2: GABAC Architecture

Given an input descriptor stream, the compression process implemented by GABAC (and specified in the MPEG-G standard) consists of a five-stage pipeline: input parsing, (optional) 3-step transformation (left grey box in Figure 1), and symbol binarization, context selection and CABAC (right grey box in Figure 1).

A. Input Parsing. In the input parsing stage, the input descriptor stream is parsed into a stream of symbols. For example, a symbol might occupy two consecutive bytes in a descriptor stream. In the input parsing stage these two bytes are then interpreted as one symbol.
The resulting stream of symbols is then processed by the 3-step transformation stage that converts the symbol stream into one or more transformed sub-streams.

B. 3-Step Transformation. In the first transformation step (depicted as box **Sequence Transformation** in Figure 1), one of three sequence transformations is selected. The sequence transformation can also be omitted. Each sequence transformation will create two or three transformed sub-streams. In the further steps of the GABAC encoding process, the transformed sub-streams are processed separately. The three available sequence transformations are:

- **Equality Coding:** The input stream is transformed into two transformed sub-streams. For each input symbol s in the input sequence, one or two output symbols are generated:
  - The first output symbol t_1^{EQ} represents an equality flag, indicating whether the symbol is equal to the previous symbol in the input sequence (1) or not (0).
  - If the input symbol is different from the previous symbol the transformation will generate a second output symbol t_2^{EQ} depending on the current input value s_n and the previous input value s_{n-1}:

$$t_2^{\text{EQ}} = \begin{cases} s_n - 1, & \text{if } s_n > s_{n-1} \\ s_n, & \text{otherwise} \end{cases}.$$

- **Run-Length Encoding (RLE) Coding:** The input stream is transformed into two transformed sub-streams. For each symbol $s_n \neq s_{n-1}$, two output symbols are generated. The first output symbol t_1^{RLE} represents the input symbol s_{n-1}. The second output symbol t_2^{RLE} represents the number of repetitions of t_1^{RLE} in retrospect until the occurrence of another symbol.

- **Match Coding:** The input stream is transformed into three transformed sub-streams. During the match coding transformation, a search window is maintained. For each new (set of) input symbol(s), the match coding transformation searches the search window for a matching block of symbols. Each set of input symbols is then replaced by two output symbols $t_{\text{length}}^{\text{M}}$ and $t_{\text{data}}^{\text{M}}$. These two output symbols are stored across three different transformed sub-streams:
  - The first output symbol $t_{\text{length}}^{\text{M}}$ represents the length of the match, where $t_{\text{length}}^{\text{M}} = 0$ means that no match has been found. This output symbol is appended to the first transformed sub-stream.
  - The second output symbol $t_{\text{data}}^{\text{M}}$ can contain one of two different types of information, depending on the value of $t_{\text{length}}^{\text{M}}$. If $t_{\text{length}}^{\text{M}} = 0$, $t_{\text{data}}^{\text{M}}$ contains a raw symbol and is appended to the second transformed sub-stream. If $t_{\text{length}}^{\text{M}} \neq 0$, $t_{\text{data}}^{\text{M}}$ contains a pointer to position of the first symbol of the match. In this case, $t_{\text{data}}^{\text{M}}$ is appended to the third transformed sub-stream.

In the second transformation step (depicted as box **LUT Transformation** in Figure 1), for each transformed sub-stream separately, a look-up table (LUT) transformation can be applied where each symbol in the transformed sub-streams is replaced by a symbol from a look-up table.
In the third transformation step, (depicted as box **Differential Transformation** in Figure 1), each LUT-transformed sub-stream can further be transformed using a differential transformation, where each symbol is replaced by the arithmetic difference to its predecessor.

The goal of this 3-step transformation process is to facilitate more effective compression when using CABAC. To achieve this goal, the 3-step transformation process converts the input symbols into sub-streams that typically exhibit:

- reduced numbers of symbols, by exploiting redundancy;
- smaller alphabets, by converting the symbols into homogeneous sub-streams;
- adapted distributions, by replacing symbols, and as such allowing for binarizations (see subsection C) that are shorter and/or offer a higher predictability;
- or a combination of aforementioned points.

An example of such a combination is match coding, followed by a LUT transformation. Match coding will typically reduce the number of symbols (a set of multiple symbols can be replaced by two symbols). The output sub-streams of the match coding transformation each contain homogeneous data (lengths, pointers, raw symbols) which can be encoded more effectively. It can generally be expected that the alphabets of these sub-streams will be smaller than that of the input symbol stream. Furthermore, if LUT transformations are applied to these sub-streams, frequent symbols will be replaced by symbols that can be represented by shorter binarization, hence limiting the number of values that have to be encoded by CABAC. Finally, it should be noted that these transformations are optional. Hence, they can be omitted if no coding effectiveness is gained when applying them.

C. Binarization. In the binarization stage (depicted as box **Binarization** in Figure 1), the symbols of each 3-step transformed sub-stream are converted into a set of bits (called *bins*). To generate this set of bins, six binarization processes are available. A binarization is selected per transformed sub-stream.

- **Binary (BI):** Each input value v is represented by its 2-base (i.e., binary) representation using a defined number of bits.

- **Truncated Unary (TU):** Each input value v is represented by v ones followed by a zero. If v is equal to a defined maximal value the trailing zero is discarded.
- **Exponential Golomb (EG)** Each input value v is represented by a prefix and a suffix. The suffix is equal to the binary representation of $v+1$. The prefix is represented by a sequence of zeros with a length equal to the length of the suffix minus 1.
- **Signed Exponential Golomb (SEG):** Negative input values $v \leq 0$ are mapped onto $-2 \cdot v$. Positive input values $v > 0$ are mapped onto $2 \cdot v$. This number is then binarized using EG.
- **Truncated Exponential Golomb (TEG):** Each input value v is represented by one or two binarizations. Input values smaller than a given value v_{max} are represented by their TU binarization. Input values larger than or equal to v_{max} are represented by the TU binarization of v_{max}, combined with the EG binarization of the input value minus v_{max}.
- **Signed Truncated Exponential Golomb (STEG):** Each input value v is represented by its TEG representation of $|v|$ and by a sign bit if $v \neq 0$.

D. Context Selection and CABAC. In parallel with the binarization stage, a context selection stage is performed (depicted as box **Context Selection** in Figure 1). A context is a number between 0 (representing a probability of 0) and 127 (representing a probability of 1). A context is used to encode a specific bin with a context-adaptive binary arithmetic encoder (CABAC), depicted as box **CABAC** in Figure 1.
Contexts are grouped into context sets. Each context set contains the contexts that are needed to support the encoding of one symbol (from a 3-step transformed sub-stream) in its binarized representation.
GABAC provides 4 different modes of context selection. In the first mode (bypass) no context selection is performed and all bins are processed using contexts that assume equiprobability. Here, contexts are not adapted. In the second mode (order-0 adaptive coding), there is one context set for each symbol. Here, the contexts are adapted. In the third (order-1 adaptive coding) and fourth mode (order-2 adaptive coding), context sets are selected based on the previously encoded symbol and based on the two previous encoded symbols, respectively. Also here, the contexts are adapted.

Supplementary Note 3: Codecs

The performance of GABAC was compared to the codecs that are used in CRAM, i.e., gzip, bzip2, xz (LZMA), rANS order-0, and rANS order-1. The rANS codec was extracted from CRAM and is publicly available at https://github.com/voges/rans.
All tests were performed on a server equipped with 2 Intel Xeon E5-2650 v3 CPUs and 128 GiB of RAM, running Ubuntu 14.04. To monitor the execution time of all codecs we used GNU time 1.7.

A. bzip2. bzip2 (http://www.bzip.org) relies on a combination of the Burrows–Wheeler transform (7) and Huffman-coding. We used the following commands to compress and decompress descriptor stream files with bzip2 (note: the -9 flag favors compression ratio over speed as much as possible):

```
$ bzip2 -9 -c test >test.bzip2
$ bzip2 -d -c test.bzip2 >test.bzip2.recon
```

B. GABAC. GABAC is the compression solution that is being analyzed in this paper. GABAC uses a combination of (optional) transformations, binarizations, context selection algorithms and context-adaptive binary arithmetic coding (CABAC) (8).
GABAC is available at https://github.com/mitogen/gabac/. For our simulations we used GABAC 1.0.0.
We used the following commands to compress and decompress descriptor stream files with GABAC:

```
$ gabacify encode -i test -o test.gabac-a
$ gabacify encode -i test -c test.gabac_configuration.json -o test.gabac
$ gabacify decode -i test.gabac -c test.gabac_configuration.json -o test.gabac.recon
```

The first command analyzes the data in the input file and infers the best possible configuration (i.e., the best possible combination of the available transformations, binarizations and context selection mechanism) using a brute force approach. The first command also encodes the input data and saves the best possible configuration in a JSON file. The second command encodes the input data with the previously found configuration. By monitoring these two commands (instead of only the first one) during the simulations, it is possible to not only measure the total execution time of GABAC, but to also measure the impact of the configuration search on the execution time. The third command decodes a GABAC bitstream given a configuration.

C. gzip. Gzip (https://www.gzip.org) is a widespread compression tool. It is utilizing the LZ77 (9) algorithm and Huffman coding (10) for compression. We used the following commands to compress and decompress descriptor stream files with gzip (note: the −9 flag favors compression ratio over speed as much as possible):

```
$ gzip -9 -c test > test.gzip
$ gzip -d -c test.gzip >test.gzip.recon
```

D. rANS. rANS is a compression algorithm based on asymmetric numeral systems (ANS) (11). We extracted the rANS code from HTSlib. Our standalone rANS executable is available at https://github.com/voges/rans. For compression and decompression using a Markov source model of order 0 we used the following commands.

```
$ rans -o0 < test >test.rans0
$ rans -d < test.rans0 >test.rans0.recon
```

For compression and decompression using a Markov source model of order 1 we used the following commands.

```
$ rans -o1 < test >test.rans1
$ rans -d < test.rans1 >test.rans1.recon
```

E. xz (LZMA). The Lempel-Ziv-Markov chain algorithm (LZMA) uses a variant of the LZ77 algorithm in combination with Markov chains and a range coder. We used the command line tool xz from the XZ Utils (https://tukaani.org/xz). We used the following commands to compress and decompress descriptor stream files with xz (note: the −9 flag favors compression ratio over speed as much as possible):

```
$ xz -9 -c test >test.xz
$ xz -d -c test.xz >test.xz.recon
```

Supplementary Note 4: Extended Results

For the set of CRAM descriptor stream files, Figure 2 shows, for each codec, the ranking distribution for the different descriptor streams along the compression axis.

For the set of DeeZ descriptor stream files, Figure 3 shows, for each codec, the ranking distribution for the different descriptor streams along the compression axis.

For the set of CRAM descriptor stream files, Figure 4 shows, for each codec, the ranking distribution for the different descriptor streams along the compression speed axis.

For the set of DeeZ descriptor stream files, Figure 5 shows, for each codec, the ranking distribution for the different descriptor streams along the compression speed axis.

For the set of CRAM descriptor stream files, Figure 6 shows, for each codec, the ranking distribution for the different descriptor streams along the decompression speed axis.

For the set of DeeZ descriptor stream files, Figure 7 shows, for each codec, the ranking distribution for the different descriptor streams along the decompression speed axis.

For figures 2, 3, 4, 5, 6 and 7 the following semantics apply:

- dots were jittered for clarity;
- the x-axes show the test set ID (01 and 02) from which the descriptor stream files were generated;
- the y-axes denote the actual ranks;
- each dot depicts the ranking a codec achieved on one specific descriptor stream file;
- the boxes visualize the following statistics: the horizontal black lines denote the medians, the lower and upper hinges correspond to the first and third quartiles, and the whiskers extend to the smallest/largest values no further than 1.5 times the inter-quartile range from the hinges;

- the dotted red lines denote the mean compression rank for each tool, averaged over both test items.

In figures 4, 5, 6 and 7 the compression and decompression ranks were computed excluding the times used for the search of the best GABAC configuration. The size of the GABAC configuration space depends on the actual input data subject to compression. For the CRAM and DeeZ descriptor streams we estimate that between 4,000 and 8,000 configurations were evaluated per descriptor stream.

In addition to the compression and decompression speed ranks, as presented in Figures 4, 5, 6 and 7, Table 4 shows the compression and decompression speeds, averaged over both sets of descriptor stream files and over all codecs. Note that the compression speed is calculated as (uncompressed size / compression time) and that the decompression speed is calculated as (compressed size / decompression time). It can be observed that in terms of compression speed GABAC, operating at an average compression speed of 19.12 MiB/s, outperforms bzip2, gzip and xz and is itself only outperformed by rANS-0 and rANS-1. In terms of decompression speed, GABAC operates at an average decompression speed of 3.67 MiB/s. Here only xz, operating at 1.83 MiB/s, is slower on average.
Nonetheless, we envision GABAC to be implemented in a genomic compression solution, were genomic data—to facilitate non-sequential access to the compressed data—is processed in atomic units. In such a scenario the compression process would very likely be highly parallelized and thus multiple GABAC instances would be executed at the same time.

Codec	Avg. comp. speed	Avg. decomp. speed
gzip	10.19 MiB/s	11.62 MiB/s
bzip2	19.12 MiB/s	3.67 MiB/s
xz	2.30 MiB/s	5.11 MiB/s
rANS-0	45.47 MiB/s	11.99 MiB/s
rANS-1	38.30 MiB/s	7.67 MiB/s
GABAC	13.27 MiB/s	1.83 MiB/s

Table 4. Compression and decompression speeds, averaged over both sets of descriptor stream files and over all codecs.

Figures 8 and 9 show the compression performance ranks versus the compression speed ranks, providing another visualization of our experiment results.

Figures 10, 11, 12 and 13 show, per data item (01, 02) and per descriptor stream set (CRAM, DeeZ) the compression ratio obtained by each codec for each descriptor stream.

Finally, all log files containing the raw experiment results are available at https://www.tnt.uni-hannover.de/staff/voges/data/gabac/. These raw results include:

- uncompressed and compressed sizes for all files and codecs;
- compression and decompression times for all files and codecs;
- compression and decompression maximum resident set sizes for all files and codecs.

Acknowledgments

The authors thank James K. Bonfield and Ibrahim Numanagić for their support in modifying CRAM and DeeZ for our experiments. The authors also thank IDLab (Ghent University - imec) for the use of their computing cluster. The authors also thank Institut für Informationsverarbeitung at Leibniz University Hannover for their support. The authors also thank the Interdisciplinary Health Sciences Institute, the Carl. R. Woese Institute for Genomic Biology and the National Center for Supercomputing Applications at the University of Illinois at Urbana-Champaign (UIUC) for their generous support and access to resources.
Special gratitude to Nathan Mattson, Steven Hart, Eric Klee, Eric Wieben, Mathieu Wiepert and Asif Hossain from Mayo Clinic and to Matthew Hudson from UIUC for their invaluable feedback, and to Gay Reed and Katherine Kendig for managing the project.
We particularly acknowledge the support of Keith Stewart, M.B., Ch.B., Mayo Clinic/Illinois Grand Challenge Sponsor and Director of the Mayo Clinic Center for Individualized Medicine.

Funding

This work has been partially supported by grants 2018-182798 and 2018-182799 from the Chan Zuckerberg Initiative DAF, a donor advised fund of the Silicon Valley Community Foundation, a Strategic Research Initiative (SRI) from UIUC and the Mayo

Clinic Center for Individualized Medicine, and the Todd and Karen Wanek Program for Hypoplastic Left Heart Syndrome. This work was a part of the Mayo Clinic and Illinois Strategic Alliance for Technology-Based Healthcare.

References

1. H Li and R Durbin. Fast and accurate short read alignment with Burrows–Wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009.
2. MA DePristo, E Banks, R Poplin, KV Garimella, JR Maguire, C Hartl, AA Philippakis, G del Angel, MA Rivas, M Hanna, A McKenna, TJ Fennell, AM Kernytsky, AY Sivachenko, K Cibulskis, SB Gabriel, D Altshuler, and MJ Daly. A framework for variation discovery and genotyping using next-generation DNA sequencing data. *Nature Genetics*, 43(5):1754–1760, 2011.
3. H Li, B Handsaker, A Wysoker, T Fennell, J Ruan, N Homer, G Marth, G Abecasis, and R Durbin. The Sequence Alignment/Map format and SAMtools. *Bioinformatics,* 25(16):2078–2079, 2009.
4. JK Bonfield. The Scramble conversion tool. *Bioinformatics*, 30(19):2818–2819, 2014.
5. M Hsi-Yang Fritz, R Leinonen, G Cochrane, and E Birney. Efficient storage of high throughput DNA sequencing data using reference-based compression. *Genome Research*, 21(5):734–740, 2011.
6. F Hach, I Numanagić, and SC Sahinalp. DeeZ: reference-based compression by local assembly. *Nature Methods*, 11(11):1082–1084, 2014.
7. M Burrows and DJ Wheeler. A Block-Sorting Lossless Data Compression Algorithm. *Digital Systems Research Center Research Reports*, 1994.
8. D Marpe, H Schwarz, and T Wiegand. Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7):620–636, 2003.
9. J Ziv and A Lempel. A Universal Algorithm for Sequential Data Compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.
10. D Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
11. J Duda. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. *arXiv*, 1311.2540, 2013.

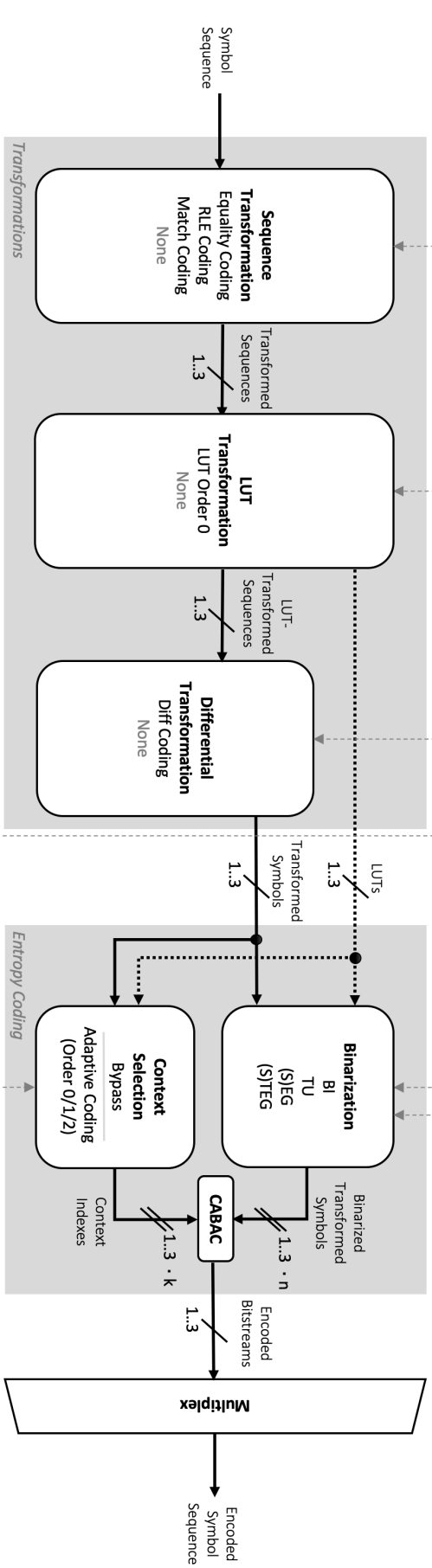

Fig. 1. Block diagram of the GABAC encoder.

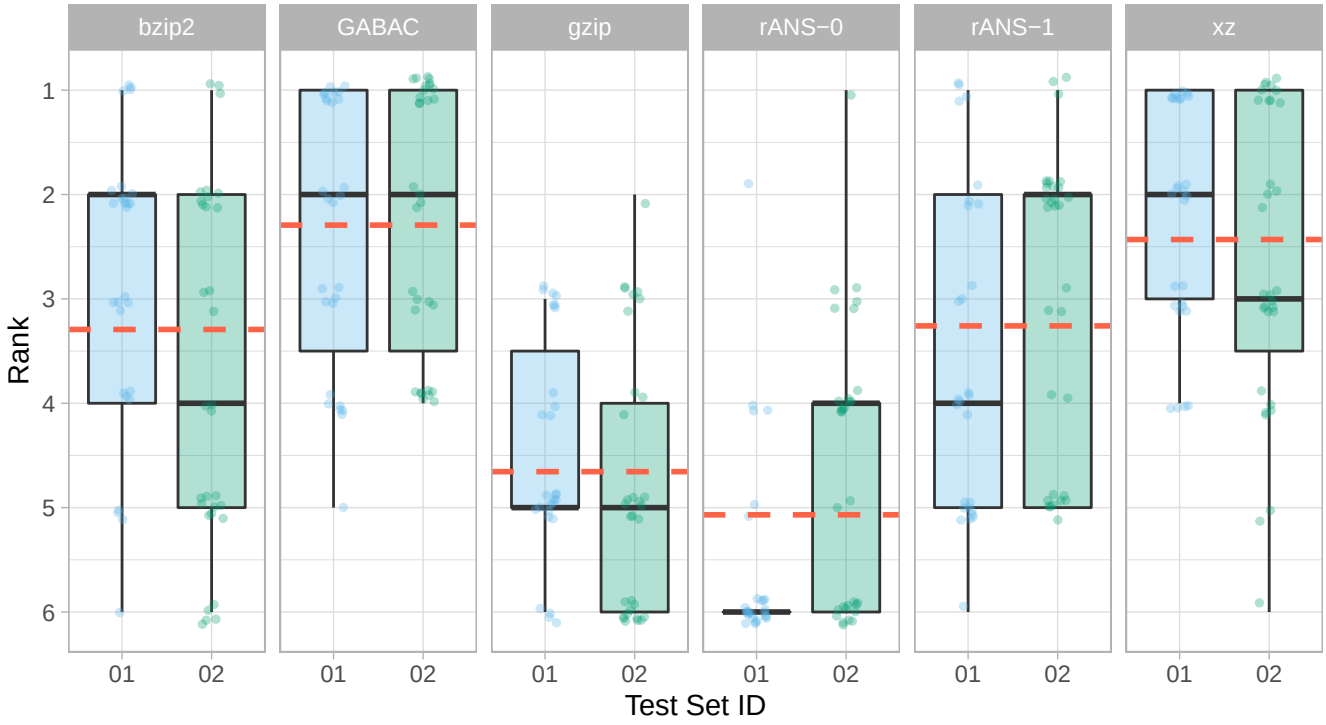

Fig. 2. Rank of compression performance for the CRAM descriptor stream files.

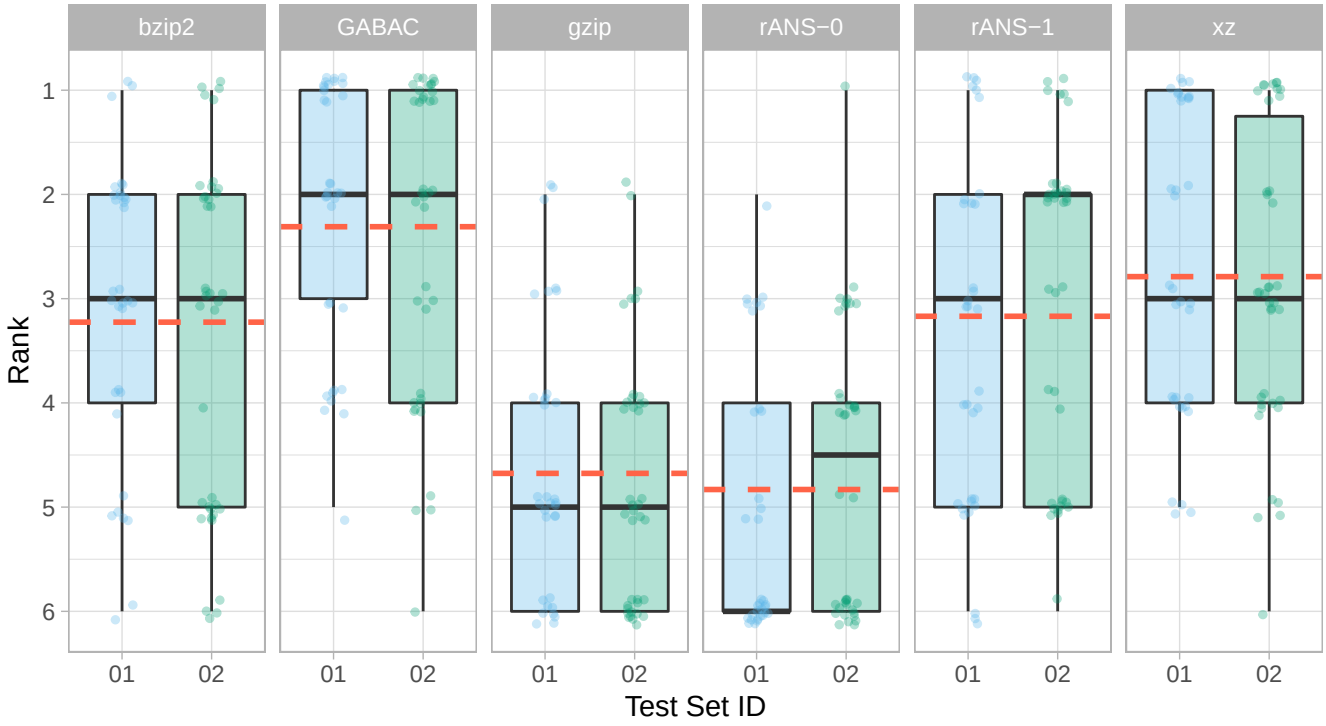

Fig. 3. Rank of compression performance for the DeeZ descriptor stream files.

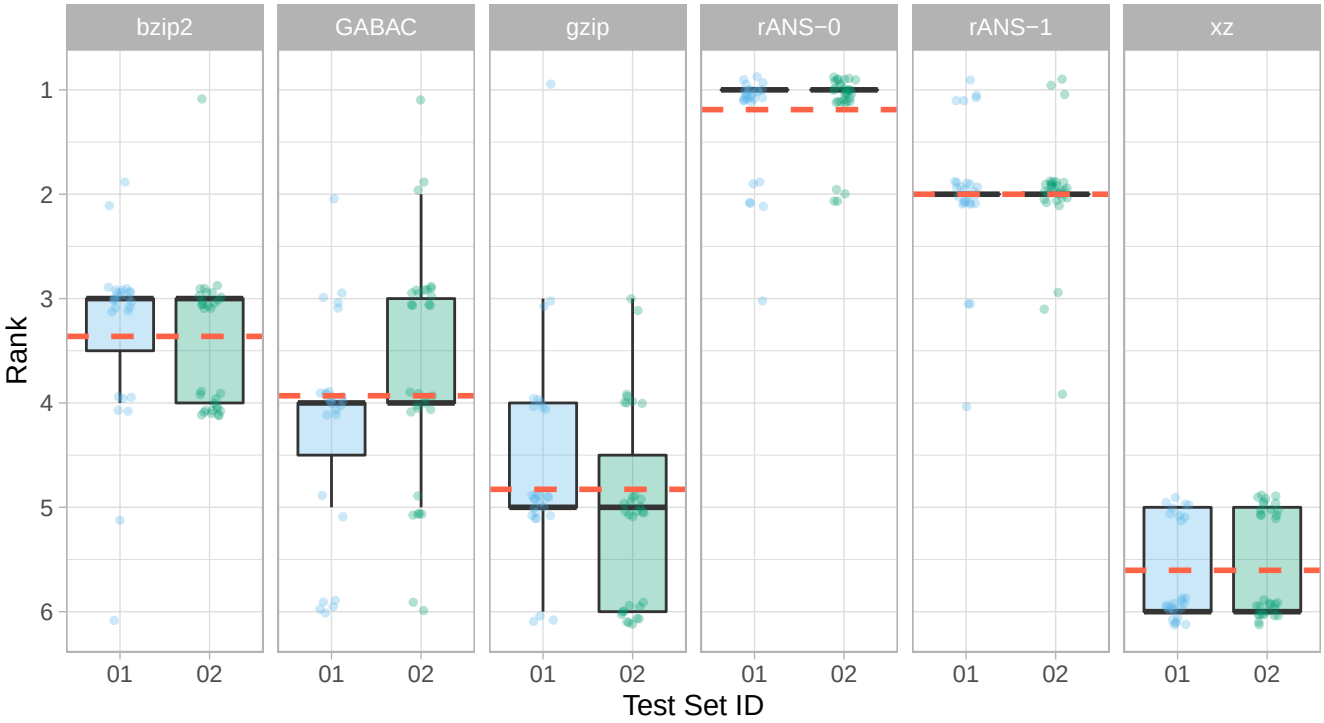

Fig. 4. Rank of compression speed performance for the CRAM descriptor stream files.

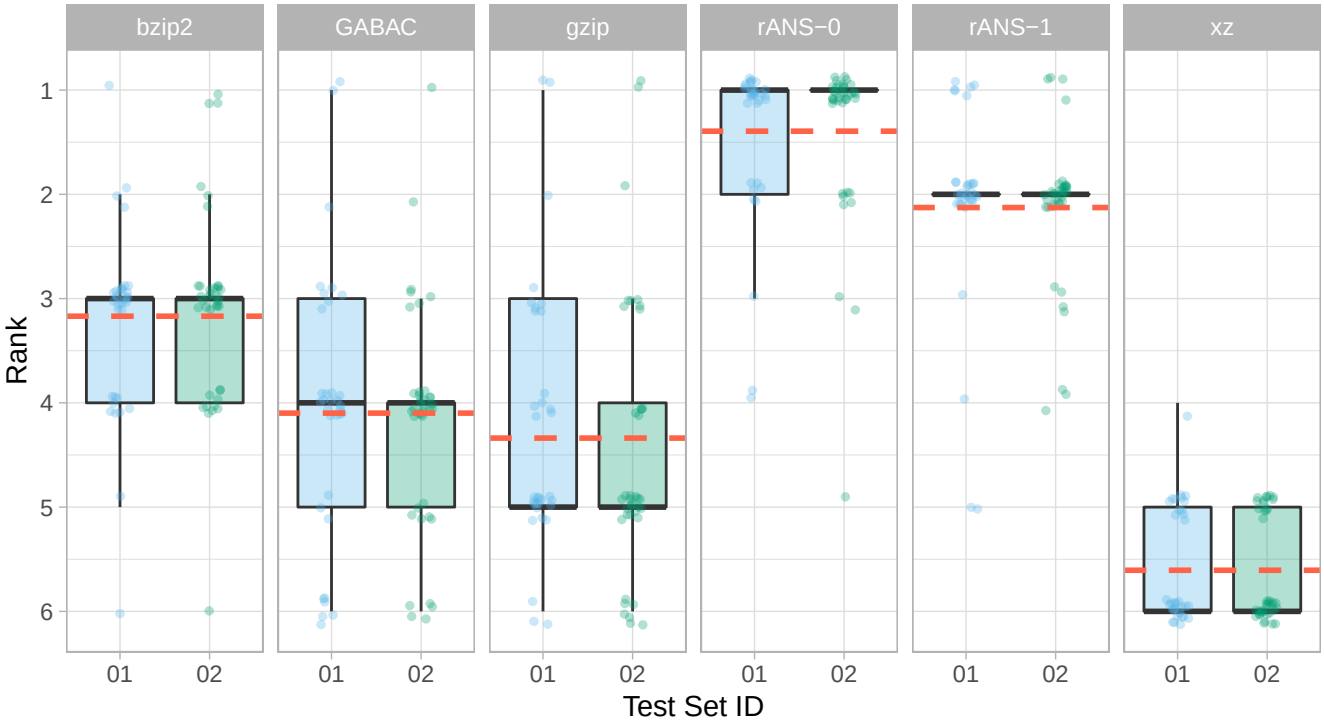

Fig. 5. Rank of compression speed performance for the DeeZ descriptor stream files.

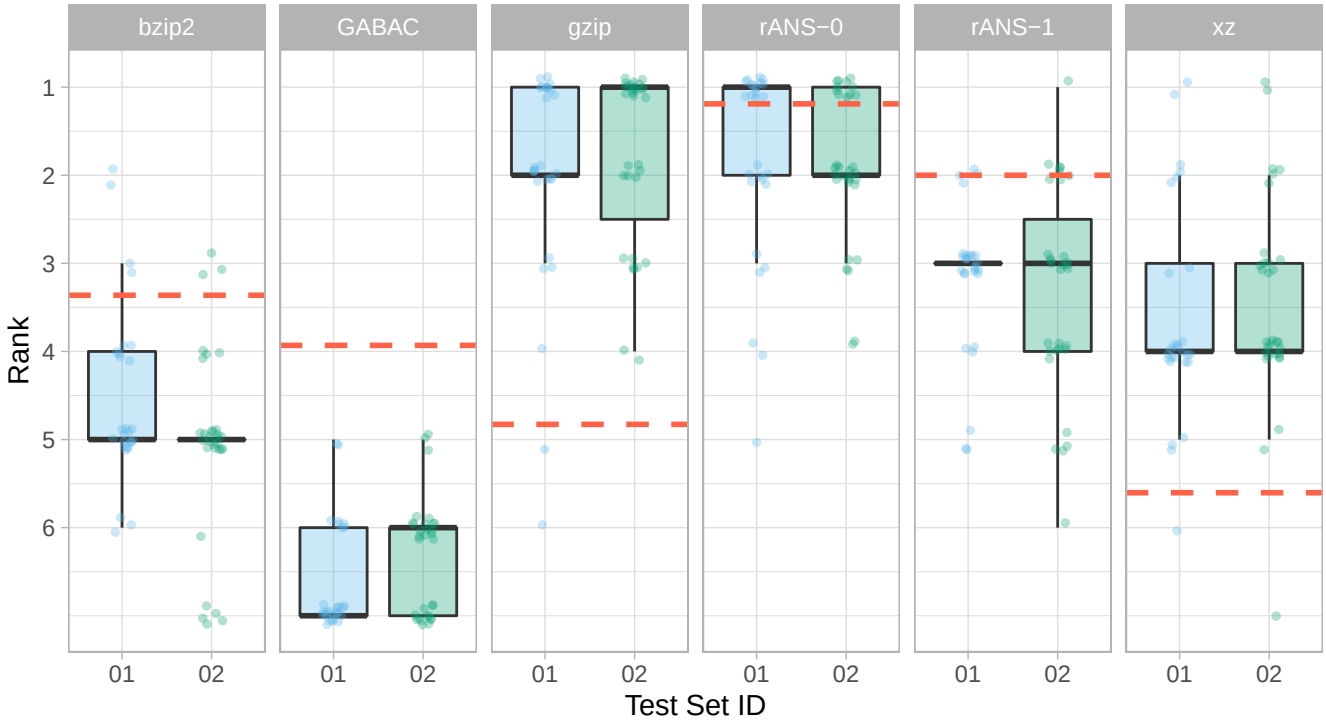

Fig. 6. Rank of decompression speed performance for the CRAM descriptor stream files.

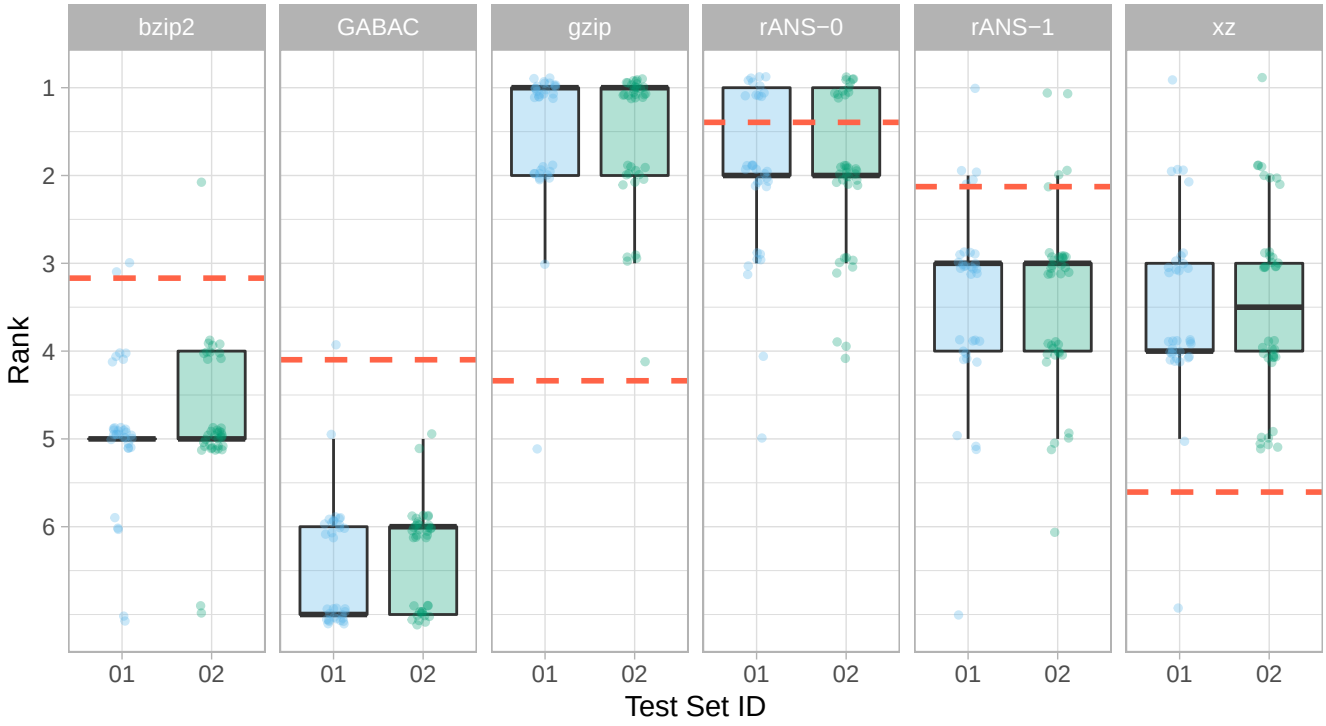

Fig. 7. Rank of decompression speed performance for the DeeZ descriptor stream files.

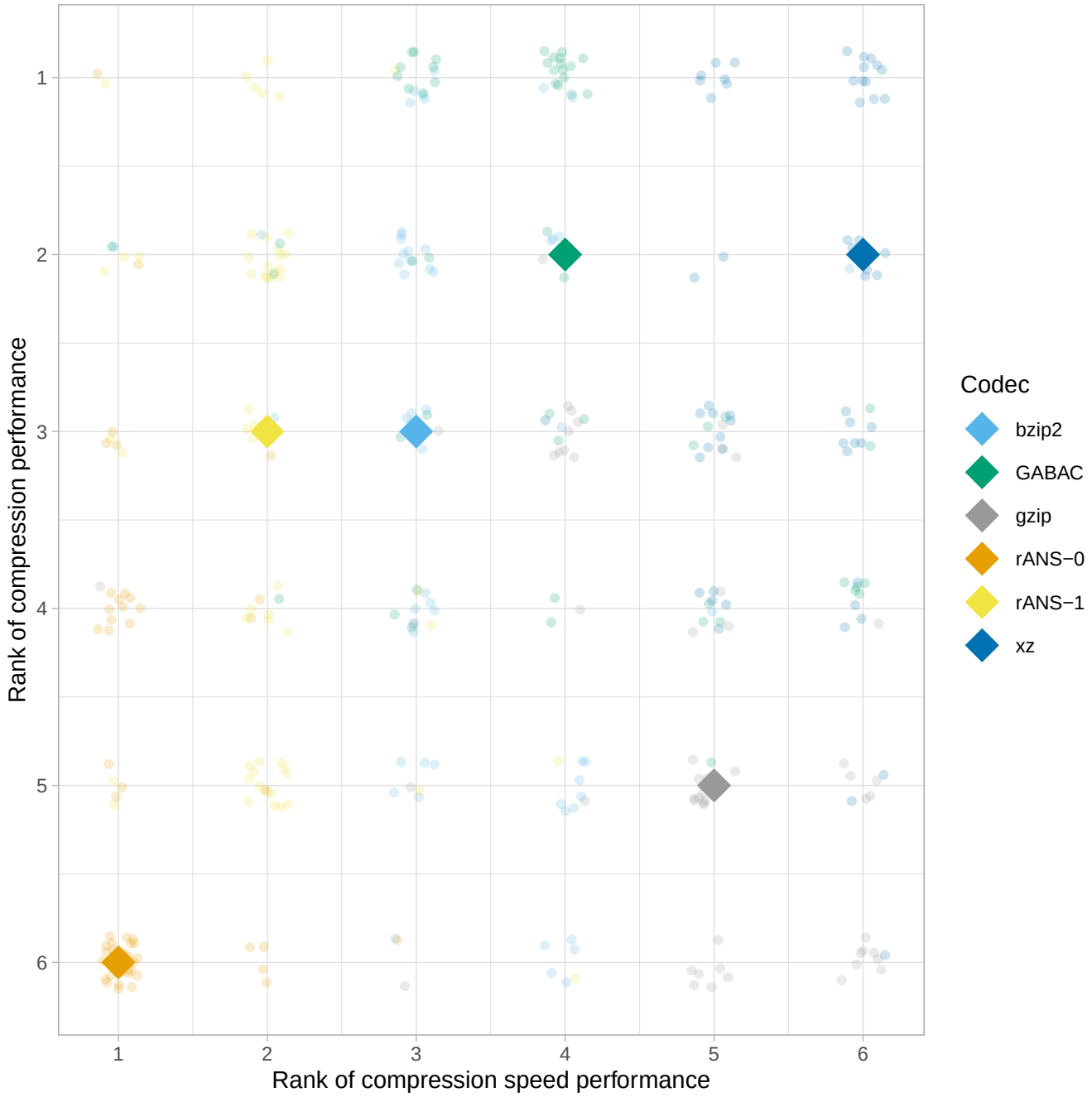

Fig. 8. Rank of compression performance versus rank of speed performance for the CRAM descriptor stream files.

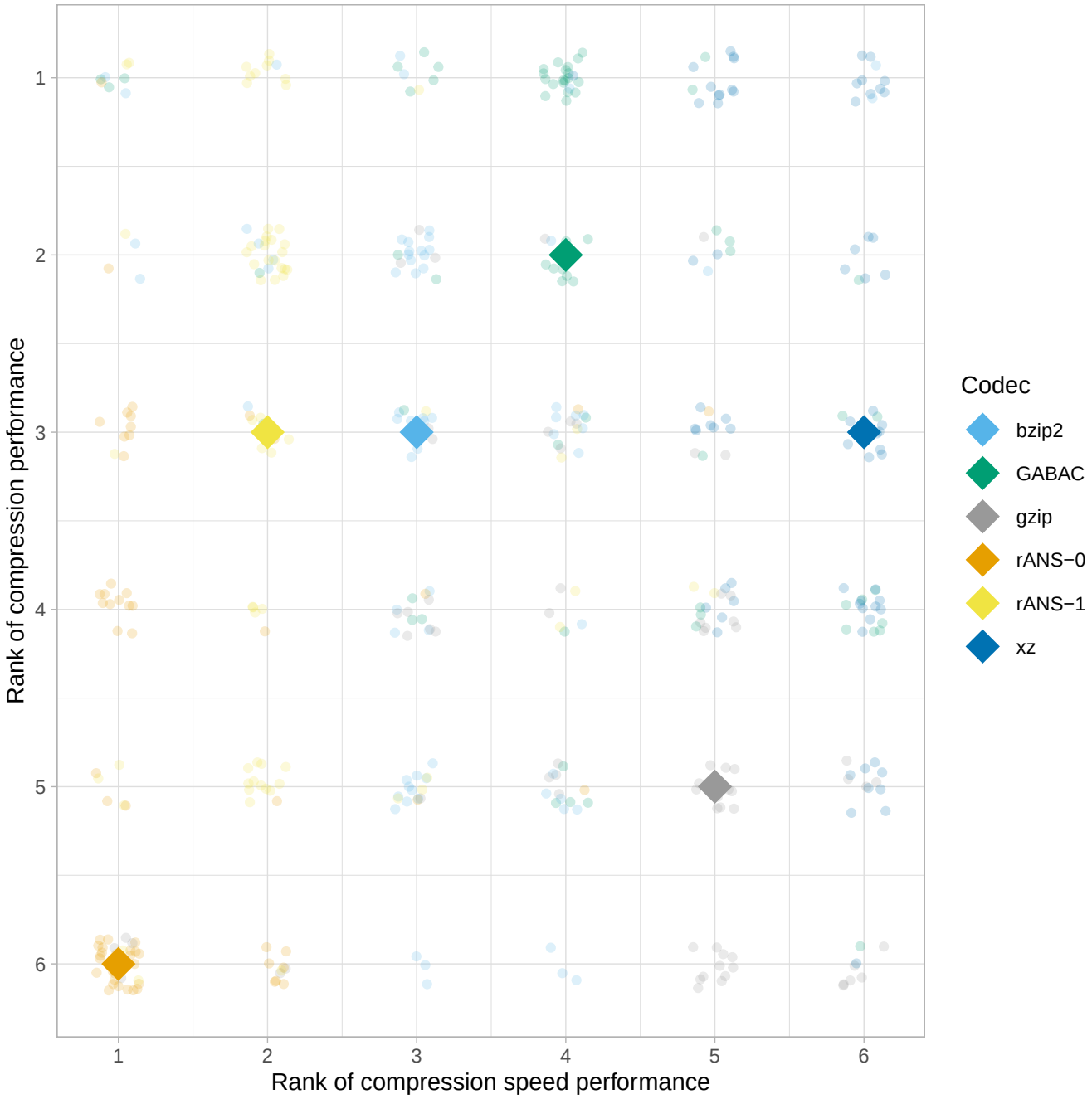

Fig. 9. Rank of compression performance versus rank of speed performance for the DeeZ descriptor stream files.

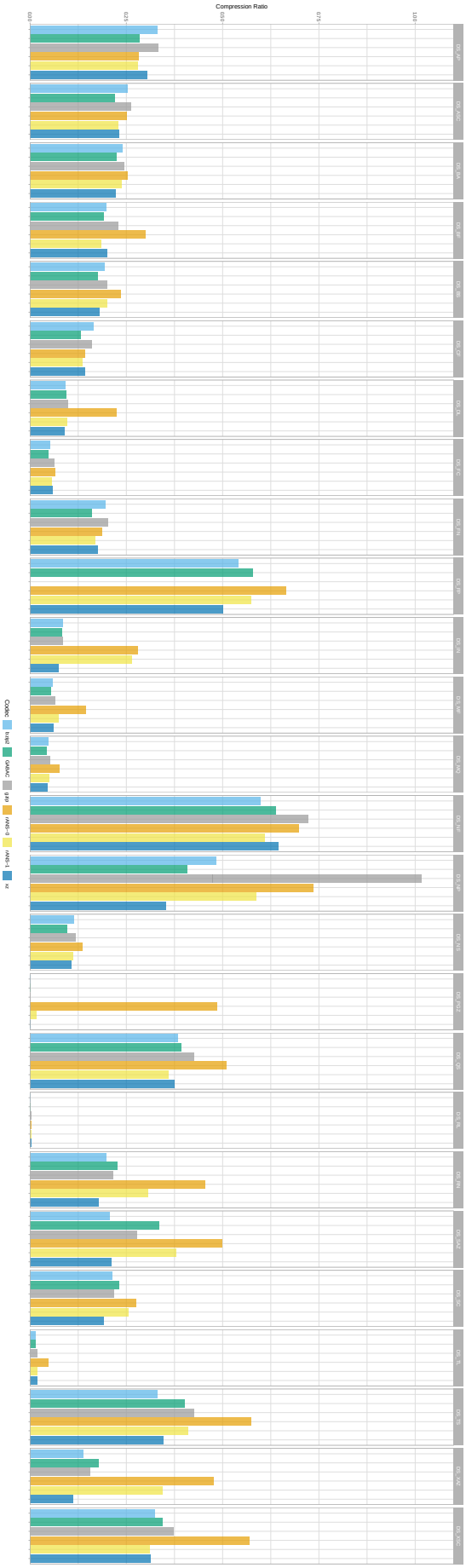

Fig. 10. Compression ratios for the CRAM descriptor stream files pertaining to item 01.

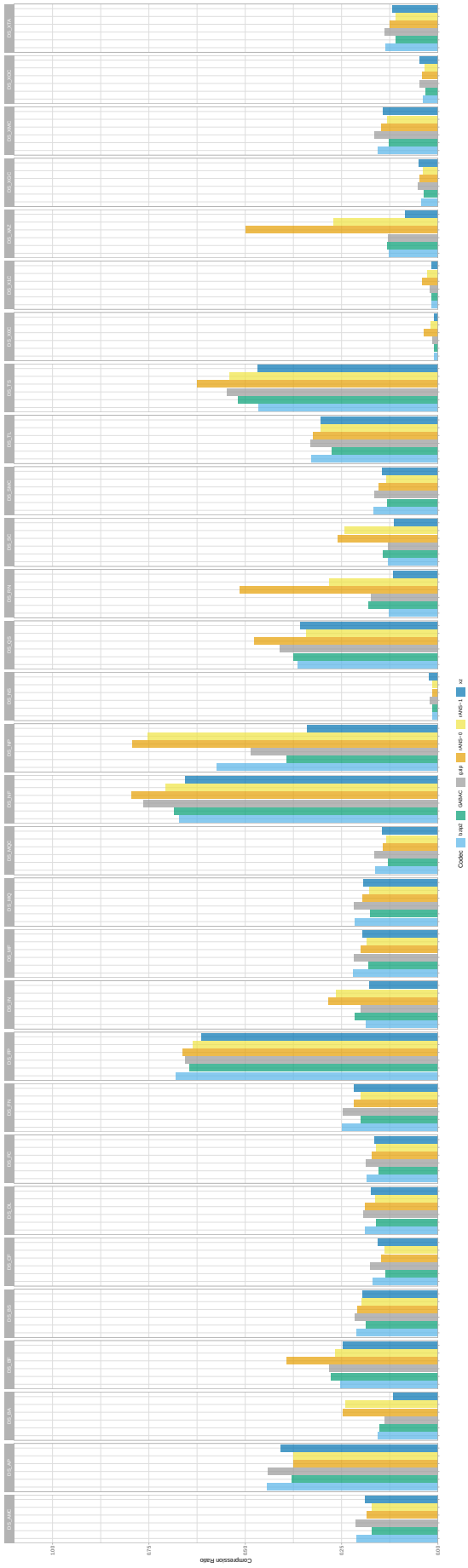

Fig. 11. Compression ratios for the CRAM descriptor stream files pertaining to item 02.

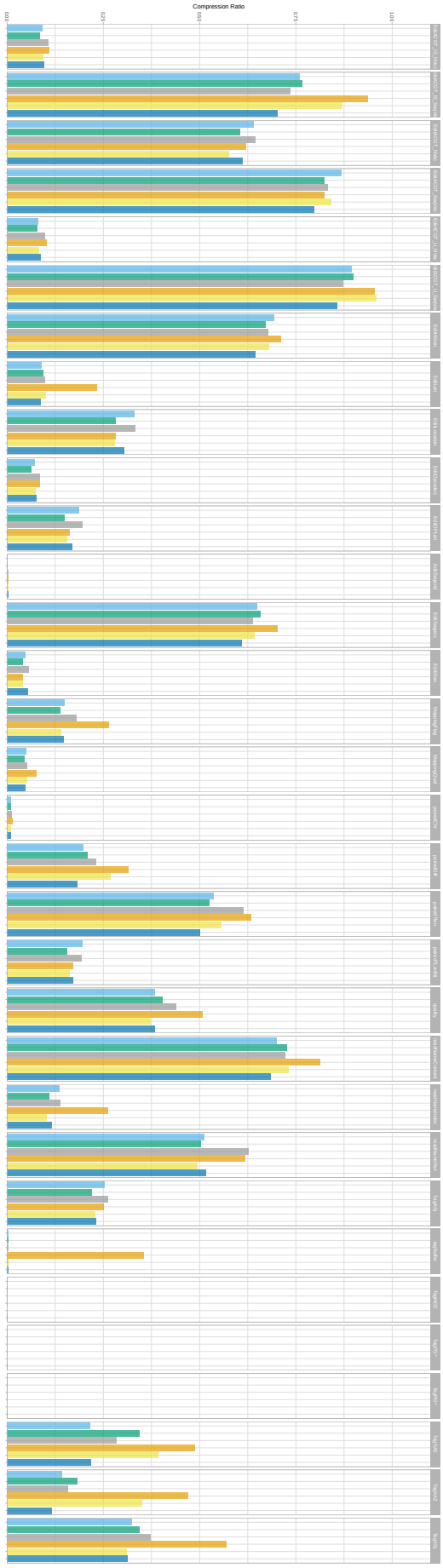

Fig. 12. Compression ratios for the DeeZ descriptor stream files pertaining to item 01.

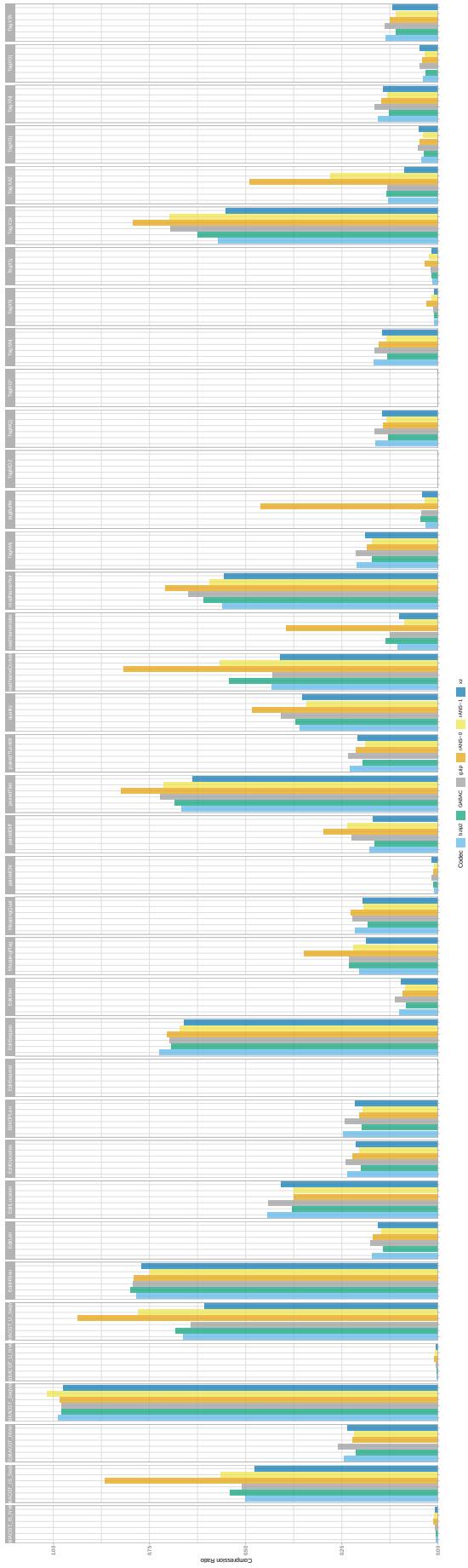

Fig. 13. Compression ratios for the DeeZ descriptor stream files pertaining to item 02.